

EXPRESS.JS

DAY 1

Express Fundamentals • Application • Routes • Request/Response • Server

Long-term reference notes

Built from the concepts, examples, practice and assessment completed during Day 1.

DAY 1 GOAL

Understand the complete basic Express flow: import Express → create the app → define a route → receive a request → run the handler → send a response → listen on a port.

Course position: Node.js is complete. This is the first day of the fixed Express.js section.

What You Learned Today

Topic	Core idea
Express.js	Node.js web framework for building servers and backend applications/APIs.
express vs app	express = imported package functionality; app = Express application instance.
Routes	HTTP method + path + handler.
req / res	Incoming request information vs outgoing response.
app.listen()	Starts listening for incoming connections on a port.
Request → Response	Client request → route match → handler → response → client.

What is Express.js?

Express.js is a web framework for Node.js. It provides a convenient way to build HTTP servers, backend applications and APIs without having to work directly with all the lower-level HTTP details every time.

```
Node.js
  ↓
Express.js
  ↓
Servers + backend applications + APIs
```

MENTAL MODEL

Node.js gives us the runtime and low-level capabilities. Express gives us a structured, convenient layer for handling routes, requests, responses and middleware.

Express as an npm Package + Importing It

Express is distributed through npm as a package. In a CommonJS Node.js project, we can load it with:

```
const express = require("express");
```

What this means: load the Express package and store what it exports in the variable named `express`.

The name `express` is a conventional variable name. The important part is what `require("express")` gives us.

TERMINOLOGY

Express = Node.js web framework.
express = the variable holding the functionality returned by importing the Express package.

express vs app

```
const express = require("express");
```

```
const app = express();
```

This distinction was the first major checkpoint of Day 1.

express	app
Imported Express functionality	Express application instance
Used to create the application	Used to configure the application

```
Express package
  ↓
express()
  ↓
Express application instance
  ↓
app
```

REMEMBER

`app.get()`, `app.use()`, `app.listen()`, etc. operate on the Express application represented by `app`.

Routes and `app.get()`

A route tells Express what to do when a particular HTTP method and path are requested.

```
app.get("/", (req, res) => {  
  res.send("Home");  
});
```

Basic structure:

```
app.get(path, handler);
```

Route = HTTP method + path + handler

```
GET + "/" + handler  
GET + "/about" + handler  
GET + "/profile" + handler
```

For example:

```
app.get("/", (req, res) => {  
  res.send("Home Page");  
});  
  
app.get("/about", (req, res) => {  
  res.send("About Page");  
});  
  
app.get("/contact", (req, res) => {  
  res.send("Contact Page");  
});
```

GET Requests

GET is an HTTP method generally used when the client wants to retrieve/request information.

When you visit a URL in a browser, the browser generally makes a GET request. For example:

```
GET /about
```

Express looks for a registered route whose HTTP method and path match the incoming request.

Route Handler

```
app.get("/about", (req, res) => {  
  res.send("About Page");  
});
```

The function below is the route handler:

```
(req, res) => {  
  res.send("About Page");  
}
```

Express calls this function when the incoming request matches the route.

```
Request matches
  ↓
Route handler executes
  ↓
Response is sent
```

Request (req) and Response (res)

CORE IDEA

req = information about what the client sent to the server.
res = the object used by the server to send a response back to the client.

```
app.get("/about", (req, res) => {
  console.log(req.method);
  console.log(req.url);

  res.send("About Page");
});
```

Examples of request information include the HTTP method and URL. Later, we will learn ``req.params``, ``req.query``, ``req.body``, and ``req.headers`` in the appropriate lessons.

Memory aid: req = incoming □ res = outgoing □

res.send()

``res.send()`` sends a response back to the client.

```
app.get("/profile", (req, res) => {
  res.send("This is my profile");
});
```

The browser displays the response text. During Day 1, you tested this route successfully in the browser.

localhost and Ports

localhost refers to the machine on which the application is running.

```
http://localhost:3000
```

Here:

localhost	The local machine / this computer
3000	The port where the application is listening

Port 3000 is common for development, but it is not mandatory. Another available port could be used.

```
app.listen(4000); // another possible development port
```

app.listen()

``app.listen()`` starts the server listening for incoming connections on the specified port.

```
app.listen(3000);
```

A common form includes a callback:

```
app.listen(3000, () => {  
  console.log("Server started");  
});
```

The callback runs after the server successfully starts listening.

DON'T CONFUSE THESE

`app.get()` → defines what to do for a matching GET request.

`app.listen()` → starts listening for incoming connections on a port.

Complete Express Application

```
const express = require("express");  
  
const app = express();  
  
app.get("/", (req, res) => {  
  res.send("Hello World");  
});  
  
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Line-by-line:

1. ``require("express")`` loads the Express package.
2. ``express()`` creates an Express application instance.
3. ``app.get("/")`` defines a GET route for the root path.
4. ``(req, res) => { ... }`` is the route handler.
5. ``res.send("Hello World")`` sends the response.
6. ``app.listen(3000)`` starts listening on port 3000.

Complete Request → Response Flow

```
Browser
|
| GET /
|
↓
Express server (port 3000)
|
↓
Find matching route
|
↓
app.get("/")
|
↓
Route handler (req, res)
|
↓
res.send("Hello World")
|
↓
Browser receives "Hello World"
```

When you type `http://localhost:3000` in the browser, it is effectively requesting the root path `/`. The server receives the GET request, Express finds `app.get("/")`, runs its handler, and the response is sent back to the browser.

THE STORY TO REMEMBER

Import Express → create app → define route → listen on port → client sends request → Express matches route → handler runs → response goes back to client.

What if the Route Doesn't Exist?

If you define routes for `/` and `/about` but request `/products`, there is no matching GET route. You will typically see:

```
Cannot GET /products
```

This means Express received the request but did not find a matching registered GET route for that path.

During Day 1, you tested this behavior yourself in the browser.

Important Distinctions

Term	Meaning
<code>express</code>	Imported Express functionality
<code>app</code>	Express application instance
<code>app.get()</code>	Registers a GET route
<code>req</code>	Incoming request information
<code>res</code>	Response object used to send data back

res.send()	Sends a response
app.listen()	Starts listening for connections
localhost	This/local machine
3000	Port number used by the local server

Common Beginner Mistakes

Confusing `express` and `app`

`express` is the imported Express functionality; `app` is the application instance we configure.

Thinking `app.get()` starts the server

`app.get()` defines a route. `app.listen()` starts listening.

Mixing up `req` and `res`

`req` represents incoming request information; `res` is used to send the response.

Thinking every URL automatically works

A path needs a matching route (or other Express handling) to produce the intended response.

Thinking port 3000 is mandatory

3000 is a common development choice, not a requirement.

Practice Completed During Day 1

You created and tested routes yourself, including:

```
app.get("/", (req, res) => {
  res.send("Hello Bhavana");
});

app.get("/profile", (req, res) => {
  res.send("This is my profile");
});

app.listen(3000, () => {
  console.log("Server started at 3000 port");
});
```

You verified /profile in the browser and received the expected response. You also tested an undefined route and observed Cannot GET /contact.

ASSESSMENT RESULT

Day 1 assessment passed. You correctly explained routes, route handlers, req, res, app.listen(), ports, and the complete request-response flow.

Day 1 Quick Recap

- Express.js is a Node.js web framework.
- `require("express")` loads the Express package.
- `express()` creates an Express application instance.
- `app` stores that application instance.
- `app.get(path, handler)` registers a GET route.
- `req` represents incoming request information.
- `res` is used to send the response.
- `res.send()` sends a response to the client.
- `localhost` means the local machine.
- `app.listen(3000)` starts listening on port 3000.
- A request must match the relevant HTTP method and path for the route handler to run.

Interview / Self-Check Questions

1. What is Express.js and why do we use it with Node.js?
2. What is the difference between ``express`` and ``app``?
3. What happens when ``const app = express()`` executes?
4. What is a route?
5. What does ``app.get()`` do?
6. What is a route handler?
7. What are ``req`` and ``res``?
8. What does ``res.send()`` do?
9. What is ``localhost``?
10. What is a port?
11. What is the difference between ``app.get()`` and ``app.listen()``?
12. Explain the complete request → route → handler → response flow.

EXPRESS.JS DAY 1 — COMPLETE

Foundation established: Express application, routes, GET requests, handlers, request/response, localhost, ports and server startup.

NEXT

Express.js Day 2 will build on this foundation. We will continue one concept at a time; no JWT, REST API, TypeORM or database topics are being mixed into Day 1.